

Block 6

Production Readiness & Operations

Secrets · Overlays · Graceful shutdown · Rolling update · Graylog · Migration

Externalized Configuration and Secrets

- Golden rule: configuration and secrets never live in the image; they are injected at start
- MSR resolves $env{NAME} in application.properties. $file{...} is NOT resolved there in 12.1
- Compose: environment: entries show up in podman inspect, so pass secrets as files instead
  - Capstone 6: Compose secrets in /run/secrets + a wrapper entrypoint that exports them, then exec startContainer.sh
  - Kubernetes: Secret → env.valueFrom.secretKeyRef, no wrapper needed
- MSR also has a built-in HashiCorp Vault connector: the target for Swisscom's real environments
- Licence: mount the file and set SAG_IS_LICENSE_FILE; handle it like a secret

Classification - Internal

Secrets Without podman inspect Exposure

compose.prod.yaml (excerpt)

```
secrets:
  admin_password: { file: ./secrets/admin_password.txt }
  db_password:    { file: ./secrets/db_password.txt }
services:
  msr-1:
    secrets: [admin_password, db_password]
    entrypoint: ["/bin/sh", "/opt/workshop/secrets-entrypoint.sh"]  # /run/secrets/x -> X
    command: ["/opt/webmethods/IntegrationServer/bin/startContainer.sh"]
    environment:
      <<: *msr-prod-env
      ADMIN_PASSWORD: !reset null      # !reset must NOT come through the anchor
      DB_PASSWORD: !reset null
```

Verified in Capstone 6: 0 passwords in podman inspect, MSR and PostgreSQL authenticate with the secret files.

Classification - Internal

Ansible Playbooks vs. Container Configuration

What Ansible does today

- Installs the product, applies fixes, lays down config files on VMs
- Templates environment values into config files
- Starts/stops services across the fleet
- Encodes institutional knowledge of a "correct install"

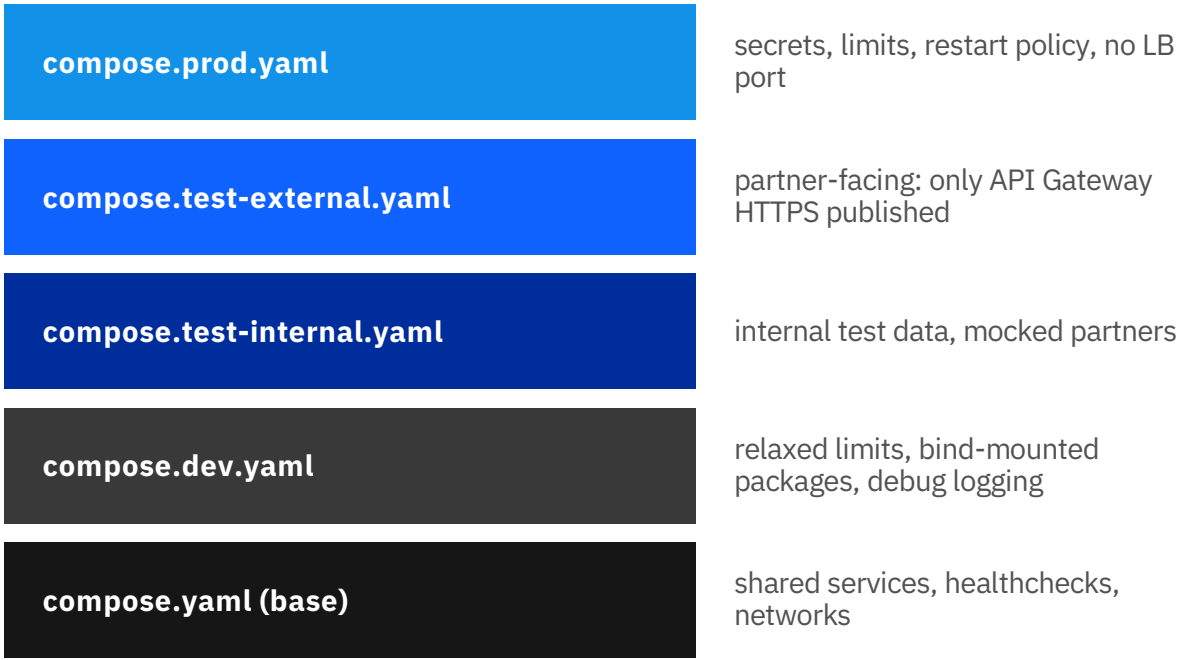
Where that moves

- Install + fixes → the image build (FROM a pinned IBM tag)
- Config templating → application.properties + $env{} + overlays
- Start/stop → container lifecycle, restart policies, rolling updates
- Ansible can still drive builds and deployments: its job changes, it doesn't disappear

Mine the playbooks for the settings they encode, then turn those settings into application.properties entries.

Classification - Internal

The 4-Environment Configuration Strategy



One base file, one thin overlay per environment, all with the SAME application.properties syntax.

GitLab CI selects the overlay with -f per deploy target.

!override replaces a value, !reset removes it.

Classification - Internal

Environment-Specific Configuration

- compose.yaml: the shared base definition, checked into Git
- compose.prod.yaml etc.: thin overlays applied with -f, only the differences
- podman-compose -f compose.yaml -f compose.prod.yaml up -d: same pattern as Helm values / Kustomize
- !reset removes an inherited value (ports: !reset []); !override replaces it (podman-compose 1.6+, Compose 2.24+)
- !reset does NOT work through a YAML merge key (<<: *anchor); write it directly on the service
- .env supplies ${VARIABLES}; commit .env.example only, never the real .env

Note: Same application.properties in every environment: overlays change environment variables, never the file's syntax.

Classification - Internal

🔧 Hands-On: Exercises 6.1–6.2

6.1	Base + prod overlay: maps merge, lists append (8443 twice!), !override replaces, !reset removes
6.2	podman inspect shows DB_PASSWORD of the env container; the secret lives in /run/secrets only
📁 **Files**	block6/exercises/overlays, block6/exercises/secrets

Participant Workbook, Block 6 · ~25 min · 🏁 you can predict the merged result before running config

Classification - Internal

Configuration Gotchas & Best Practices

Gotchas & pitfalls

- ✗ Copy-pasted full Compose files per environment drift within weeks
- ✗ environment: secrets are visible in podman inspect and process listings
- ✗ $file{...} in application.properties is taken literally: the password becomes the string
- ✗ !reset inside a YAML anchor is silently ignored

Best practices

- ✓ Base + thin overlays; diff the rendered config (compose config) in CI
- ✓ Secrets as files or vault, exported to the process only
- ✓ $env{} in application.properties, one file for all environments
- ✓ Check podman inspect for passwords as part of the release checklist

Classification - Internal

🏆 Capstone 6: Production-Ready , Part 1: Secrets/Prod Overlay

Objectives

≈ 90 min

- Part 1: secret files, compose.prod.yaml, entrypoint wrapper; 0 passwords in podman inspect
- Limits, restart on-failure:5, stop_grace_period 90 s, nginx port closed
- *Part 2: drain → graceful stop → re-create 1.1.0 → healthy → undrain, node by node*
- *curl-loop through the Gateway: 0 failed requests*
- *Part 3: Fluent Bit sidecar → Graylog; search by source, logtype, environment*
- ⭐ *check_msr.sh as a Nagios/OP5 check*

Prerequisites

- Capstone 5 stack running
- Image 1.1.0 built (Exercise 5.2)
- secrets/ via make-secrets.sh --from-env
- Reference: block5/solution/ incl. RUNBOOK

Deliverable
compose.yaml + compose.prod.yaml, your rolling-update runbook, the curl-loop summary with 0 failed, a Graylog screenshot with both nodes.

Classification - Internal

Graceful Shutdown: Measured in Capstone 6

Stop timeout	Time to stop	Exit code	Result
podman stop (default 10 s)	~11 s	137	SIGKILL before MSR finished: in-flight work lost
stop_grace_period: 90s	~13 s	0	"Server shutdown completed": clean

startContainer.sh traps SIGTERM and runs shutdown.sh, which waits up to SAG_IS_SHUTDOWN_WAIT_TIME (default 300 s). Set stop_grace_period above that value.

Classification - Internal

Container Lifecycle Management

- restart: on-failure:5: restarts a crash, surfaces a permanent misconfiguration after 5 attempts
- stop_grace_period: 90s + SAG_IS_SHUTDOWN_WAIT_TIME=60 so MSR drains before the engine kills it
- PID 1 must receive SIGTERM: wrapper entrypoints must exec the start script (Capstone 6's secrets wrapper does)
- Repeated restarts on the same error are a config or dependency problem, not a transient blip
- Liveness for restart decisions, readiness for traffic decisions

Classification - Internal

🔧 Hands-On: Exercise 6.3

6.3	sleep as PID 1 ignores SIGTERM: 10 s, exit 137. With --init: 1 s, exit 143. MSR needs ~13 s
📁 Files	No files needed: alpine:3.19

Participant Workbook, Block 6 · ~10 min · 🏁 you can explain stop_grace_period: 90s

Classification - Internal

Rolling Update with Compose: One Node at a Time

Drain	Stop	Recreate	Wait	Undrain
mark msr-2 down in nginx, reload	graceful, ~13 s	new image tag, --no-deps	healthy (readiness), ~65–75 s	back in nginx, reload; repeat for msr-1

Measured in Capstone 6: 0 failed out of 99 requests while both nodes moved from v1.0.0 to v1.1.0.

Edit upstream.conf in place: sed -i swaps the file and a bind mount can briefly see it missing.

Classification - Internal

Zero Downtime: What Compose Can and Can't Do

- Compose has no rolling-update orchestration: compose up with a new tag stops, then starts
- Pragmatic pattern: N ≥ 2 nodes behind nginx, update one at a time (scripts/rolling-update.sh)
- nginx resolves service names on reload: always reload after recreating a node
- podman-compose re-creates a service's dependents: nginx must NOT depends_on the MSR nodes
- proxy_next_upstream retries the other node if one dies mid-request
- Manual and single-host: exactly the gap a Kubernetes Deployment closes

Note: Compose gets you a production-representative architecture; automated zero-downtime across hosts is Kubernetes' job.

Classification - Internal

🔧 Hands-On: Capstone 6, Part 2: Rolling Update

Drain	Append ' down' to msr-2 in nginx/upstream.conf (editor, not sed -i), reload nginx
Stop	time podman-compose … stop msr-2: graceful, exit 0, ~13 s
Update	APP_VERSION=1.1.0 … up -d --no-deps --no-build msr-2; wait for healthy (~70 s)
Undrain	Remove ' down', reload (re-resolves the new IP); repeat for msr-1; set APP_VERSION in .env
📁 Files	block5/capstone/nginx/upstream.conf, block5/capstone/scripts/curl-loop.sh; reference: block5/solution/scripts/rolling-update.sh

Participant Workbook, Block 6 · ~35 min · 🏁 curl-loop summary: 0 failed

Classification - Internal

Logging with Graylog

- The Docker gelf log driver ships stdout to Graylog, but Podman does NOT support it ("invalid log driver")
- Portable pattern (Capstone 6): a Fluent Bit sidecar tails MSR's logs volume and sends GELF/UDP to Graylog
- The sidecar also ships file-only logs (WMSECURITY_* audit) that stdout never contains
- Enrich every record: source (node), logtype (server/security), environment
- LOG_FORMAT=json on MSR gives structured fields; Graylog 6.1 + OpenSearch 2.x as the workshop back end

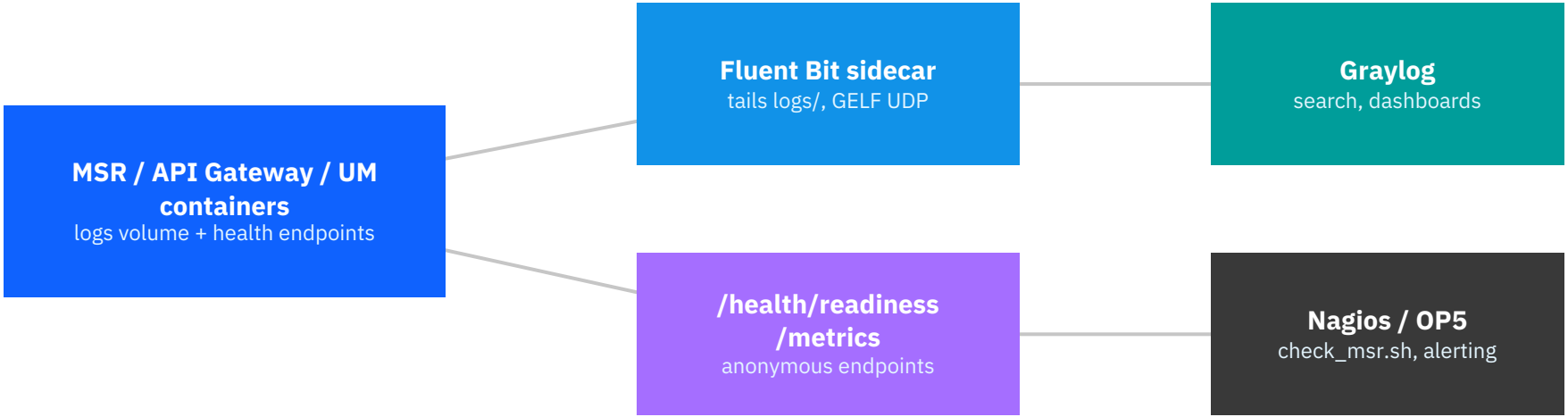
Classification - Internal

Monitoring with Nagios / OP5

- Poll the same signal the containers use: /health/readiness (anonymous, 503 lists DOWN indicators)
- Performance data from /metrics: sag_is_used_memory_bytes / sag_is_max_memory_bytes, sag_is_server_threads
- scripts/check_msr.sh: a Nagios-compatible plugin (OK / WARNING / CRITICAL, perfdata) built in Capstone 6
- Alert on readiness transitions, restart counts, heap pressure, pool saturation, UM store size
- Goal: the existing OP5 dashboards watch the container fleet from day one

Classification - Internal

Observability Around the Container Fleet



Logs flow to Graylog for search and forensics; health and metrics flow to OP5 for alerting.

Classification - Internal

Operations Gotchas & Best Practices

Gotchas & pitfalls

- ✗ Default 10 s stop timeout: MSR gets SIGKILLed (exit 137)
- ✗ Planning the gelf log driver on Podman hosts: it isn't supported
- ✗ podman logs alone is useless at 24+ instances
- ✗ Container CPU/memory only: misses JVM heap pressure and thread exhaustion

Best practices

- ✓ stop_grace_period > SAG_IS_SHUTDOWN_WAIT_TIME
- ✓ Fluent Bit sidecar (or DaemonSet on K8s), LOG_FORMAT=json
- ✓ Ship logs centrally from Dev onwards
- ✓ Alert on readiness + /metrics heap and threads

Classification - Internal

🔧 Hands-On: Capstone 6, Part 3: Logs in Graylog

Start	… -f compose.logging.yaml up -d; ./graylog/create-gelf-input.sh once
Search	localhost:9010 (admin/admin): source:msr-1, logtype:security, environment:capstone5
⭐	check_msr.sh via exec -T against both nodes; provoke WARNING with -w 5
🗂️ **Files**	block5/capstone/compose.logging.yaml, block5/capstone/fluent-bit/, block5/capstone/graylog/, block5/capstone/scripts/check_msr.sh

Participant Workbook, Block 6 · ~25 min · 🏁 both nodes' messages arrive within ~15 s

Classification - Internal

💬 Q&A and Discussion: Capstone 6

1. *Your rolling update had 0 failed requests: what's missing before ops can run that runbook, or GitLab can?*

2. *Secrets left podman inspect: where do Swisscom's passwords live today, and what would feed the secret files?*

3. *Graylog via Fluent Bit, OP5 via /health and /metrics: what changes in today's logging and monitoring?*

Classification - Internal

Dos and Don'ts for webMethods Containers

Area	Do	Don't
Images	Pin IBM tags/digests, scan every build	Use latest, skip scanning
Config	One application.properties with $env{}	Different files or hand edits per environment
Secrets	Files/vault exported to the process	environment: entries, ARGs, COPY into layers
Startup	service_healthy + /health/readiness	depends_on alone, liveness as readiness
Shutdown	stop_grace_period 90 s	The default 10 s stop timeout (exit 137)
Sizing	JAVA_MAX_MEM ≈ 65–75 % of the limit	Unset heap in a capped container
Networking	Service names, segmented networks	localhost or hard-coded IPs
Ops	Central logs + readiness alerts	podman logs on 24 hosts

Classification - Internal

Security & Resource Management

- IBM images already run as UID 1724 (sagadmin); keep USER 1724 in every Containerfile
- Set deploy.resources.limits (CPU, memory) and size the heap to it (JAVA_MIN_MEM = JAVA_MAX_MEM ≈ 65–75 %)
- JDK 21 (OpenJ9) is container-aware: the cgroup limit is respected; leave headroom for class metadata, threads, direct memory
- Scan continuously: new CVEs appear for images that passed their original scan
- Segment networks: Gateway public, MSR/UM/DB internal, admin consoles not published in prod
- Restrict the health/metrics ACLs if they're reachable beyond the internal network

Classification - Internal

Windows → Linux Migration Strategy

What changes

- Paths: backslashes → forward slashes; case-sensitive filesystem
- Line endings CRLF → LF in flat files, scripts, resources
- Native adapter libraries (.dll) need Linux (.so) equivalents
- Windows services/scheduled tasks → container lifecycle + scheduler
- NTFS ACLs → POSIX permissions (UID 1724)

How to de-risk it

- Inventory every package for Windows paths and OS calls first
- Run flat-file/EDI packages through a Linux dry run in Dev
- Confirm Linux support of every third-party adapter/driver early
- Migrate environment by environment: Dev → Test-int → Test-ext → Prod
- Keep Windows running in parallel until Linux has a full traffic cycle

Classification - Internal

Migration Gotchas & Best Practices

Gotchas & pitfalls

- ✗ A Windows path separator fails only when one specific code path runs
- ✗ CRLF-sensitive parsing passes tests and fails on a real partner file
- ✗ An adapter without a Linux driver blocks its packages, found too late
- ✗ Cutting over all 4 environments at once removes the comparison baseline

Best practices

- ✓ Package-by-package inventory before the first Containerfile
- ✓ Line-ending and path test cases in the Dev validation suite
- ✓ Verify adapter/driver Linux support in week 1 of the project
- ✓ Migrate and test one environment at a time with a fallback

Classification - Internal

High Availability: What Compose Can and Can't Do

- Compose can: run N MSR replicas behind a load balancer, share UM/DB, restart failed containers
- Compose cannot: survive a host failure, schedule across hosts, roll updates without scripting
- Genuine HA at Swisscom's scale: Kubernetes Deployments (MSR, Gateway), StatefulSets (UM, TN/AT), Services, a multi-node scheduler
- IBM publishes Helm charts for MSR, UM and API Gateway: github.com/IBM/webmethods-helm-charts
- The optional Kubernetes engagement picks up exactly at this boundary

Note: Everything in Capstones 5–6 maps 1:1 onto Kubernetes objects; today's Compose work is the blueprint, not throwaway.

Classification - Internal

Podman Compose vs Kubernetes

	Podman Compose	Kubernetes
Hosts	Single	Multi-Host Cluster
Declaration / Deployment	Simple YAML	Complex manifests (Deployment, Service, etc.)
Scaling	Manual scaling	Auto-scaling (HPA)
Self-Healing	No self-healing	Self-healing & rolling updates
Best Fit	Dev/Test focus	Production-grade orchestration

Classification - Internal

Compose Concept → Kubernetes Concept (Outlook)

Compose (Capstones 5–6)	Kubernetes	Why it matters
msr-1/msr-2 (anchor)	Deployment (replicas: 2) + Service	Multi-node scheduling, automatic rolling updates
healthcheck /health/readiness	readinessProbe (+ liveness on /health/liveness)	Traffic gating and self-healing
stop_grace_period: 90s	terminationGracePeriodSeconds: 90	Same SIGTERM contract
application.properties mount	ConfigMap	Same file, same $env{} syntax
secrets + wrapper entrypoint	Secret → secretKeyRef env	No wrapper needed
um + volume	StatefulSet + PVC	Stable identity and storage
nginx + rolling-update.sh	Service / Ingress + RollingUpdate strategy	Fully automated

Full mapping in block6/solution/stretch/compose-to-kubernetes.md.

Classification - Internal

💬 Q&A and Discussion: Lessons Learned

1. *Which Dos and Don'ts conflict with how the Windows/VM environment works today?*
2. *Where do Swisscom's 400 packages split into images and release units?*
3. *In which order do we migrate the 4 environments, and what's the rollback per step?*
4. *Which open questions go into the Expert Labs follow-up documentation?*

Classification - Internal

Next Steps for Swisscom

- IBM Expert Labs follow-up (Day 5): documentation, code repository, reference architecture, migration roadmap
- Decide the package-to-image split and the first environment to migrate
- Decide the secrets approach (MSR's HashiCorp Vault connector or CyberArk) before the first non-Dev deployment
- Set up the GitLab pipeline (build → Trivy → push) against the Swisscom registry
- Review the Graylog/OP5 integration once the containerized Dev environment is live
- Revisit the optional Kubernetes engagement after a full Test cycle on Compose

> **Best practice:** This deck, the Participant Workbook and the workshop repository with all solutions are yours to keep and extend.

Classification - Internal

Workshop Recap: What You Can Do Now

- ✓ Run, configure and troubleshoot Microservices Runtime containers with application.properties
- ✓ Design and debug multi-container Compose stacks with correct startup ordering
- ✓ Build multi-stage custom images and run them through GitLab CI with a security gate
- ✓ Run API Gateway, UM, TN/AT and JDBC workloads with the right storage, networking and secrets
- ✓ Operate the fleet: graceful shutdown, rolling updates, Graylog, OP5
- ✓ A risk-managed Windows → Linux migration path for Swisscom

Coming up next: Day 5 follow-up deliverables from IBM Expert Labs, and when ready, the optional Kubernetes engagement.

Classification - Internal



Thank you

Questions & Open Discussion

IBM webMethods Expert Labs · David Poland

Swisscom contact: Karl Gilen (Karl.Gilen@swisscom.com)

Materials & code: workshop repository

Follow-up: Day 5 hand-off